INDEX

Sr No	Practical	Date	Sign
1	Develop a secure messaging application where users can exchange messages securely using RSA encryption. Implement a mechanism for generating RSA key pairs and encrypting/decrypting messages.		
2	Allow users to create multiple transactions and display them in an organised format.		
3	Create a Python class named Transaction with attributes for sender, receiver, and amount. Implement a method within the class to transfer money from the sender's account to the receiver's account.		
4	Implement a function to add new blocks to the miner and dump the blockchain.		
5	Write a python program to demonstrate mining.		
6	Write a Solidity program that demonstrates various types of functions including regular functions, view functions, pure functions, and the fallback function.		
7	Write a Solidity program that demonstrates function overloading, mathematical functions, and cryptographic functions.		
8	Write a Solidity program that demonstrates various features including contracts, inheritance, constructors, abstract contracts, interfaces.		
9	Write a Solidity program that demonstrates use of libraries, assembly, events, and error handling.		

Practical 1

Aim :- Develop a secure messaging application where users can exchange messages securely using RSA encryption. Implement a mechanism for generating RSA key pairs and encrypting/decrypting messages.

Code :

```
# Install cryptography if not already installed
!pip install cryptography

from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
import base64

# Function to generate RSA key pairs
def generate_rsa_key_pair():
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    public_key = private_key.public_key()
    return private_key, public_key

# Function to serialize keys
def serialize_keys(private_key, public_key):
    priv_pem = private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption()
    )
    pub_pem =
        public_key.public_bytes( encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo
    )
    return priv_pem, pub_pem

# Encrypt a message using public key
def encrypt_message(public_key, message):
    encrypted = public_key.encrypt(
        message.encode(),
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return base64.b64encode(encrypted).decode()
# Decrypt a message using private key
def decrypt_message(private_key, encrypted_message):
    decrypted = private_key.decrypt(
        base64.b64decode(encrypted_message),
        padding.OAEP(
```

```
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
    return decrypted.decode()

# Simulating two users: Alice and Bob
print("Generating RSA key pairs for Alice and Bob...")
alice_private, alice_public = generate_rsa_key_pair()
bob_private, bob_public = generate_rsa_key_pair()

# Alice sends a message to Bob
message_from_alice = "Hi Bob, this is Alice. The message is secure!"
print("\nOriginal message from Alice:", message_from_alice)

encrypted_message = encrypt_message(bob_public, message_from_alice)
print("Encrypted message (sent to Bob):", encrypted_message)

# Bob decrypts the message
decrypted_message = decrypt_message(bob_private, encrypted_message)
print("Decrypted message by Bob:", decrypted_message)
```

Output:

```
Requirement already satisfied: cryptography in /usr/local/lib/python3.11/dist-packages
(43.0.3)
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.11/dist-packages (from
cryptography) (1.17.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from
cffi>=1.12->cryptography) (2.22)
Generating RSA key pairs for Alice and Bob...
Original message from Alice: Hi Bob, this is Alice. The message is secure!
Encrypted message (sent to Bob):
KGlFiU+Y9e1x0C0prVI1CLMr0hHQZqYygLv/EcXM2Sp8vSBeG7bgcK0u4/+UgpT+UmK
vdOaH+NC/pIemG4yPINiaYx3G+Uh9GTTHJDpDWvCxdqjEDvdYTJVQE2uKxgvLLPNc
OhenKxogCriQqG81biZnofxXpbq225k9UFlIVvKTcdAb52PSTsPTOy6vWso+HrkhmnIAH
gbd0E/FJVh68KuiIQdZ8mmu1U7E9qqOOGot6qPsQZAdL0sHyc3Kl9kgLSLerZ3lwn3KcU
Fekb5hbXyqOCI3jgxP7L8a1zuDvusWxhThy1/+T194y9+Mj7m2haX3iOONXTfXx1B3Dbv
B8Q==
Decrypted message by Bob: Hi Bob, this is Alice. The message is secure!
```

Practical 2

Aim:- Allow users to create multiple transactions and display them in an organised format.

Code:

```
!pip install pandas

import pandas as pd

transactions = []

def create_transaction(sender, receiver, amount, description=""):
    transaction = {
        "Sender": sender,
        "Receiver": receiver,
        "Amount": amount,
        "Description": description
    }
    transactions.append(transaction)
    print("✅ Transaction recorded successfully!")


def display_transactions():
    if transactions:
        df = pd.DataFrame(transactions)
        print("\n📋 All Transactions:")
        display(df)
    else:
        print("⚠️ No transactions found.")

# Example usage
create_transaction("Alice", "Bob", 150, "Payment for services")
create_transaction("Bob", "Charlie", 75, "Dinner split")
create_transaction("Charlie", "Alice", 25, "Coffee refund")

# Display all transactions
display_transactions()
```

Output:

```
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages (2.2.2)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/dist-packages
(from pandas) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-
packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from
pandas) (2025.2)
```

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Transaction recorded successfully!

Transaction recorded successfully!

Transaction recorded successfully!

All Transactions:

	Sender	Receiver	Amount	Description
0	Alice	Bob	150	Payment for services
1	Bob	Charlie	75	Dinner split
2	Charlie	Alice	25	Coffee refund

Practical 3

Aim:- Create a Python class named Transaction with attributes for sender, receiver, and amount. Implement a method within the class to transfer money from the sender's account to the receiver's account.

Code:

```
# Simulate user accounts with balances
accounts = {
    "Alice": 500,
    "Bob": 300,
    "Charlie": 200
}

# Transaction class definition
class Transaction:
    def _init_(self, sender, receiver, amount):
        self.sender = sender
        self.receiver = receiver
        self.amount = amount

    def transfer(self):
        # Check if users exist
        if self.sender not in accounts or self.receiver not in accounts:
            print("❌ Either sender or receiver account doesn't exist.")
            return

        # Check for sufficient funds
        if accounts[self.sender] < self.amount:
            print(f"❌ Insufficient funds in {self.sender}'s account.")
            return

        # Perform transfer
        accounts[self.sender] -= self.amount
        accounts[self.receiver] += self.amount
        print(f"☑ {self.amount} transferred from {self.sender} to {self.receiver}.")

# Example usage
print("    Initial account balances:", accounts)

# Create transactions
t1 = Transaction("Alice", "Bob", 100)
t1.transfer()

t2 = Transaction("Bob", "Charlie", 50)
t2.transfer()

t3 = Transaction("Charlie", "Alice", 300)  # This should fail due to insufficient funds
```

```
t3.transfer()

print("\n💼 Final account balances:", accounts)
```

Output:

```
Initial account balances: {'Alice': 500, 'Bob': 300, 'Charlie': 200}
✅ 100 transferred from Alice to Bob.
✅ 50 transferred from Bob to Charlie.
❌ Insufficient funds in Charlie's account.

💼 Final account balances: {'Alice': 400, 'Bob': 350, 'Charlie': 250}
```

Practical 4

Aim:- Implement a function to add new blocks to the miner and dump the blockchain.

Code:

```
import hashlib
import time
import json

# Block class
class Block:
    def _init_(self, index, previous_hash, timestamp, data, nonce=0):
        self.index = index
        self.previous_hash = previous_hash
        self.timestamp = timestamp
        self.data = data
        self.nonce = nonce
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        block_string =
f"{self.index}{self.previous_hash}{self.timestamp}{json.dumps(self.data)}{self.nonce}"
        return hashlib.sha256(block_string.encode()).hexdigest()

# Blockchain class
class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 4  # Number of leading zeros in the hash

    def create_genesis_block(self):
        return Block(0, "0", time.time(), "Genesis Block")

    def get_latest_block(self):
        return self.chain[-1]

    def mine_block(self, data):
        previous_block = self.get_latest_block()
        index = previous_block.index + 1
        timestamp = time.time()
        nonce = 0

        print(f"⛏ Mining block #{index}...")

        new_block = Block(index, previous_block.hash, timestamp, data, nonce)
        while not new_block.hash.startswith('0' * self.difficulty):
            new_block.nonce += 1
            new_block.hash = new_block.calculate_hash()
```

```
        self.chain.append(new_block)
        print(f"✅ Block #{index} mined: {new_block.hash}")

    def dump_chain(self):
        print("\n📦 Blockchain Dump:") for
        block in self.chain:
            print({
                'Index': block.index,
                'Previous Hash': block.previous_hash,
                'Timestamp': time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime(block.timestamp)),
                'Data': block.data,
                'Nonce': block.nonce,
                'Hash': block.hash
            })

# Create the blockchain
my_blockchain = Blockchain()

# Add (mine) blocks
my_blockchain.mine_block({"sender": "Alice", "receiver": "Bob", "amount": 50})
my_blockchain.mine_block({"sender": "Bob", "receiver": "Charlie", "amount": 25})
my_blockchain.mine_block({"sender": "Charlie", "receiver": "Alice", "amount": 10})

# Dump the entire blockchain
my_blockchain.dump_chain()
```

Outpput:

```
⛏️ Mining block #1...
✅ Block #1 mined:
0000037cc25274fa244cc6e1af1fdd6afa8aa6fd901d5c3889e9372b88e1c9ac
⛏️ Mining block #2...
✅ Block #2 mined:
00002bae30c62e2b6c2194ad790c0988af3a787656194d1023ee8bb20bc4eb68
⛏️ Mining block #3...
✅ Block #3 mined:
00007bd9b89e93898220a22f0b1674089558dccaed8b0e4ea54c8217175fb424
📦 Blockchain Dump:
{'Index': 0, 'Previous Hash': '0', 'Timestamp': '2025-05-07 03:01:35', 'Data': 'Genesis Block',
'Nonce': 0, 'Hash': '5b0087d56d98b0a8caf97c28ee28ff434bc984c9708842361d5a0725a0b15fac'}
{'Index': 1, 'Previous Hash':
'5b0087d56d98b0a8caf97c28ee28ff434bc984c9708842361d5a0725a0b15fac', 'Timestamp':
'2025-05-07 03:01:35', 'Data': {'sender': 'Alice', 'receiver': 'Bob', 'amount': 50}, 'Nonce':
11545, 'Hash': '0000037cc25274fa244cc6e1af1fdd6afa8aa6fd901d5c3889e9372b88e1c9ac'}
{'Index': 2, 'Previous Hash':
'0000037cc25274fa244cc6e1af1fdd6afa8aa6fd901d5c3889e9372b88e1c9ac', 'Timestamp':
```

'2025-05-07 03:01:36', 'Data': {'sender': 'Bob', 'receiver': 'Charlie', 'amount': 25}, 'Nonce': 37227, 'Hash': '00002bae30c62e2b6c2194ad790c0988af3a787656194d1023ee8bb20bc4eb68'}
{'Index': 3, 'Previous Hash': '00002bae30c62e2b6c2194ad790c0988af3a787656194d1023ee8bb20bc4eb68', 'Timestamp': '2025-05-07 03:01:36', 'Data': {'sender': 'Charlie', 'receiver': 'Alice', 'amount': 10}, 'Nonce': 100290, 'Hash': '00007bd9b89e93898220a22f0b1674089558dccaed8b0e4ea54c8217175fb424'}

Practical 5

Aim:- Write a python program to demonstrate mining.

Code:

```
import hashlib
import time

# Define the mining function
def mine_block(block_data, difficulty):
    prefix_str = '0' * difficulty
    nonce = 0
    start_time = time.time()

    print("⛏ Starting mining...")
    while True:
        text = f"{block_data}{nonce}"
        hash_result = hashlib.sha256(text.encode()).hexdigest()
        if hash_result.startswith(prefix_str):
            end_time = time.time()
            print(f"✅ Block mined successfully!")
            print(f"🔢 Nonce: {nonce}")
            print(f"🔐 Hash: {hash_result}")
            print(f"⏱ Time taken: {end_time - start_time:.2f} seconds") break
        nonce += 1

# Example usage
block_data = "Alice pays Bob 10 BTC"
difficulty = 4  # Increase for higher difficulty

mine_block(block_data, difficulty)
```

Output:

```
⛏ Starting mining...
✅ Block mined successfully!
   Nonce: 2040
   Hash: 000077330197cd1a3f60c19a7990fa2b8ee23911e7d378d6e37d7d5d11d10b21
⏱ Time taken: 0.01 seconds
```

Practical 6

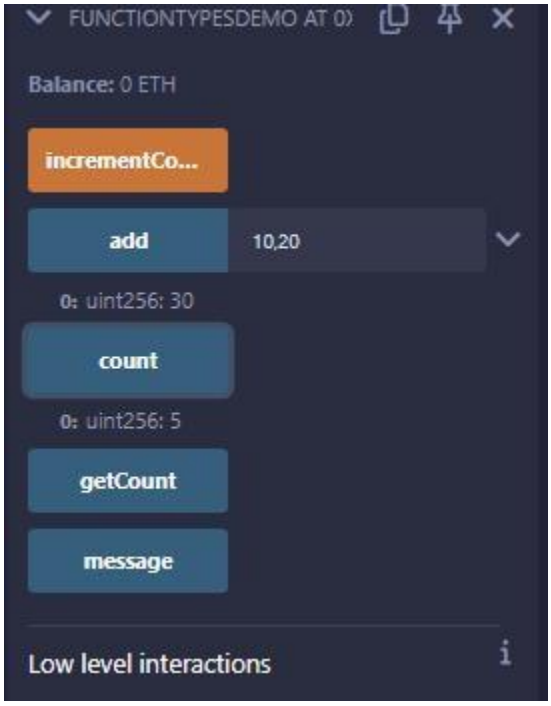
Aim:- Write a Solidity program that demonstrates various types of functions including regular functions, view functions, pure functions, and the fallback function.

Code:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

contract FunctionTypesDemo

    { uint public count;

    string public message;

    constructor() {

        count = 0;

        message = "Hello, Ethereum!";

    }

    // Regular function: modifies the contract state

    function incrementCount() public {

        count += 1;

    }

    // View function: reads state but does not modify it

    function getCount() public view returns (uint) {

        return count;

    }

    // Pure function: does not read or modify state

    function add(uint a, uint b) public pure returns (uint) {

        return a + b;

    }

    // Fallback function: called when no other function matches or when plain ether is sent

    fallback() external payable {

        message = "Fallback function called";
```

```
    }
    // Receive function: explicitly handles plain Ether transfers
            receive() external payable {
        message = "Receive function called";
    }
}
```

Output



Practical 7

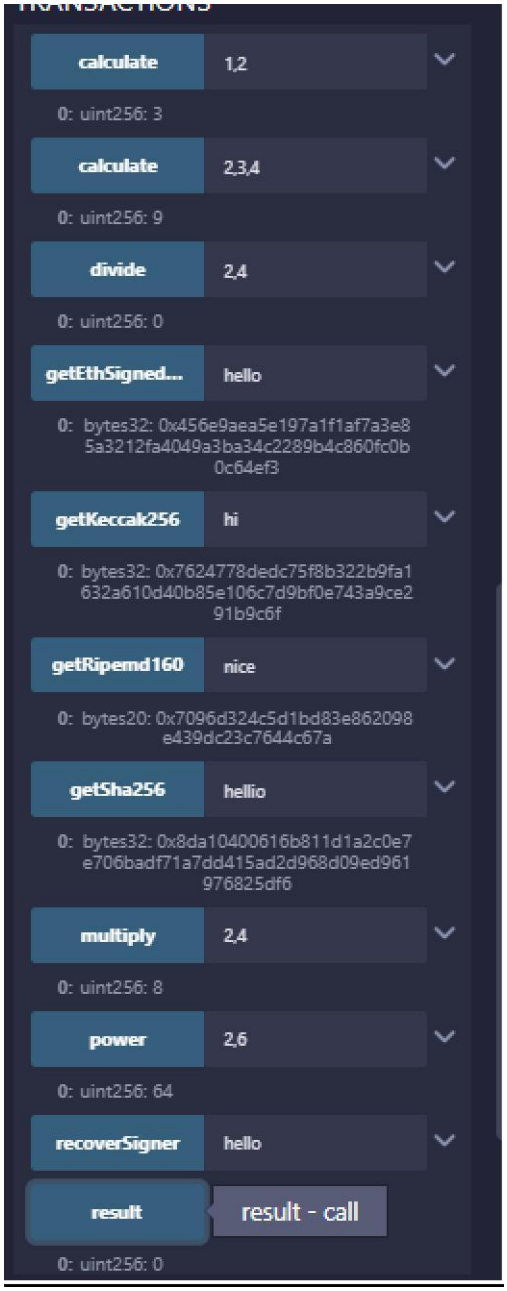
Aim: Write a Solidity program that demonstrates function overloading, mathematical functions, and cryptographic functions.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract AdvancedFunctionDemo {
    // State variable
    uint public result;
    // --- Function Overloading ---
    // Overload 1: adds two numbers
    function calculate(uint a, uint b) public pure returns (uint)
        { return a + b;
    }
    // Overload 2: adds three numbers
    function calculate(uint a, uint b, uint c) public pure returns (uint)
        { return a + b + c;
    }
    // --- Mathematical Operations ---
    function multiply(uint a, uint b) public pure returns (uint)
        { return a * b;
    }
    function divide(uint a, uint b) public pure returns (uint)
        { require(b != 0, "Division by zero");
       return a / b;
    }
    function power(uint base, uint exponent) public pure returns (uint)
        { return base ** exponent;
    }
    // --- Cryptographic Hash Functions ---
    function getKeccak256(string memory input) public pure returns (bytes32) {
```

```
        return keccak256(abi.encodePacked(input));
    }
    function getSha256(string memory input) public pure returns (bytes32)
        { return sha256(abi.encodePacked(input));
    }
    function getRipemd160(string memory input) public pure returns (bytes20)
        { return ripemd160(abi.encodePacked(input));
    }
    // --- ecrecover Example (Signature Verification) ---
    // This recovers the signer address from a message and its signature
    function recoverSigner(bytes32 messageHash, uint8 v, bytes32 r, bytes32 s) public pure
returns (address) {
        return ecrecover(messageHash, v, r, s);
    }
    // Utility function to hash a message in Ethereum style (with prefix)
    function getEthSignedMessageHash(string memory message) public pure returns (bytes32)
{
        bytes32 msgHash = keccak256(abi.encodePacked(message));
        return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", msgHash));
    }
}
```

Output:


calculate
1,2
0: uint256: 3
calculate
2,3,4
0: uint256: 9
divide
2,4
0: uint256: 0
getEthSigned...
hello
0: bytes32: 0x456e9aea5e197a1f1af7a3e85a3212fa4049a3ba34c2289b4c860fc0b0c64ef3
getKeccak256
hi
0: bytes32: 0x7624778dedc75f8b322b9fa1632a610d40b85e106c7d9bf0e743a9ce291b9c6f
getRipemd160
nice
0: bytes20: 0x7096d324c5d1bd83e862098e439dc23c7644c67a
getSha256
hellio
0: bytes32: 0x8da10400616b811d1a2c0e7e706badf71a7dd415ad2d968d09ed961976825df6
multiply
2,4
0: uint256: 8
power
2,6
0: uint256: 64
recoverSigner
hello
result
result - call
0: uint256: 0

Practical 8

Aim: Write a Solidity program that demonstrates various features including contracts, inheritance, constructors, abstract contracts, interfaces.

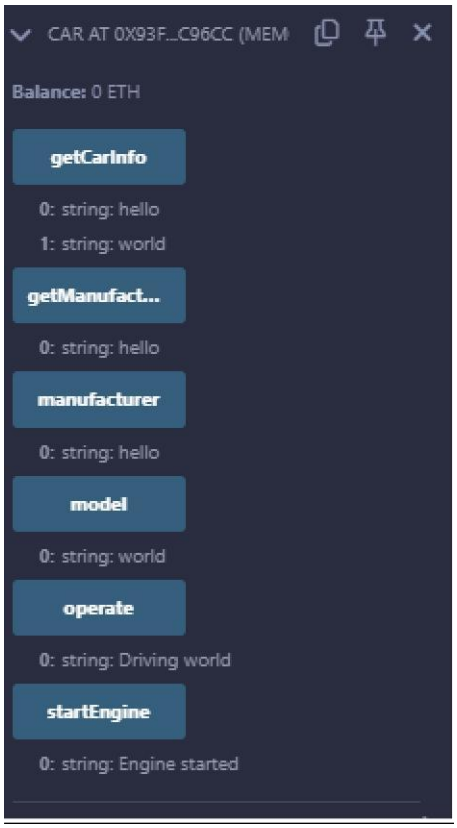
Code:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

// ------------------------------------------

// Interface: Defines external interaction

// ------------------------------------------

interface IVehicle {

    function startEngine() external pure returns (string memory);

}

// ------------------------------------------

// Abstract Contract: Provides base logic

// ------------------------------------------

abstract contract Machine

    { string public manufacturer;

    constructor(string memory _manufacturer)

        { manufacturer = _manufacturer;

    }

    function getManufacturer() public view returns (string memory)

        { return manufacturer;

    }

    // Abstract method to be implemented in derived contracts

    function operate() public view virtual returns (string memory);

}

// ------------------------------------------

// Base Contract: Implements abstract and interface

// ------------------------------------------

contract Car is Machine, IVehicle

    { string public model;
```

```
    constructor(string memory _manufacturer, string memory _model)
        Machine(_manufacturer)
    {
        model = _model;
    }
    // Implementing abstract method
    function operate() public view override returns (string memory)
        { return string(abi.encodePacked("Driving ", model));
    }
    // Implementing interface method
    function startEngine() public pure override returns (string memory)
        { return "Engine started";
    }
    function getCarInfo() public view returns (string memory, string memory)
        { return (manufacturer, model);
    }
}

// ------------------------------------------
// Derived Contract: Inherits from Car
// ------------------------------------------
contract ElectricCar is Car
    { uint public batteryLevel;
    constructor(
        string memory _manufacturer,
        string memory _model,
        uint _batteryLevel
    ) Car(_manufacturer, _model)
        { batteryLevel = _batteryLevel;
    }
```

```
    function recharge() public
        { batteryLevel = 100;
    }
    function getElectricCarInfo()
        public
        view
        returns (string memory, string memory, uint)
    {
        return (manufacturer, model, batteryLevel);
    }
}
```

Output :



Practical 9

Aim: Write a Solidity program that demonstrates use of libraries, assembly, events, and error handling.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// ---------------------------------
// Library: Utility functions
// ---------------------------------
library MathLib {
    function square(uint x) internal pure returns (uint)
        { return x * x;
    }
    function cube(uint x) internal pure returns (uint)
        { return x * x * x;
    }
}

// ---------------------------------
// Main Contract
// ---------------------------------
contract AdvancedFeatures
    { using MathLib for uint;

    uint public result;

    // ----------------------------
    // Events
    // ---------------------------
    event Computed(string operation, uint value);
```

```
event ErrorHandled(string reason);
// ---------------------------
// Custom Error
// --------------------------
error DivisionByZero();
error UnderflowError(uint a, uint b);

// --------------------------
// Function using a Library
// --------------------------
function computeSquare(uint x) public
    { result = x.square();
    emit Computed("square", result);
}
function computeCube(uint x) public
    { result = x.cube();
    emit Computed("cube", result);
}
// --------------------------
// Function with Inline Assembly
// --------------------------
function multiplyAssembly(uint a, uint b) public returns (uint product)
    { assembly {
        product := mul(a, b)
    }
    result = product;
    emit Computed("assemblyMultiply", result);
}
// --------------------------
// Function with Error Handling
```

```
    // ----------------------------
    function safeDivide(uint a, uint b) public returns (uint)
        { if (b == 0) {
            emit ErrorHandled("Division by zero attempted");
            revert DivisionByZero();
        }
        result = a / b;
        emit Computed("safeDivide", result);
        return result;
    }
    function safeSubtract(uint a, uint b) public returns (uint)
        { if (b > a) {
            emit ErrorHandled("Underflow detected");
            revert UnderflowError(a, b);
        }
        result = a - b;
        emit Computed("safeSubtract", result);
        return result;
    }
}
```

Output :

DEEP LEARNING

INDEX

Sr. No.	Practical Description	Pg. No.	Date	Sign
1	**Introduction to TensorFlow**			
a	Create tensors with different shapes and data types. Perform basic operations like addition, subtraction, multiplication, and division on tensors. Reshape, slice, and index tensors to extract specific elements or sections. Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow	1		
b	Program to solve the XOR problem.	5		
2	**Linear Regression**			
a	Implement a simple linear regression model using TensorFlow's low-level API (or `tf.keras`). Train the model on a toy dataset (e.g., housing prices vs. square footage). Visualize the loss function and the learned linear relationship. Make predictions on new data points.	6		
3	**Convolutional Neural Networks (Classification)**			
a	Implementing deep neural network for performing binary classification task	8		
b	Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.	11		
4	Write a program to implement deep learning Techniques for image segmentation.	13		
5	Write a program to predict a caption for a sample image using LSTM.	14		
6	Applying the Autoencoder algorithms for encoding real-world data	16		
7	Write a program for character recognition using RNN and compare it with CNN.	18		
8	Denoising of images using autoencoder.	20		
9	Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.(google stock price)	22		
10	Applying Generative Adversarial Networks for image generation and unsupervised tasks.	24		

Practical No.1 A

Introduction to TensorFlow

I] Create tensors with different shapes and data types

```
import tensorflow as tf
# Scalar
scalar = tf.constant(5)

# Vector (1D)
vector = tf.constant([1, 2, 3], dtype=tf.int32)

# Matrix (2D)
matrix = tf.constant([[1.0, 2.0],
                      [3.0, 4.0]], dtype=tf.float32)

# 3D Tensor
tensor_3d =
   tf.constant([ [[1, 2], [3,
   4]],
   [[5, 6], [7, 8]]
 ], dtype=tf.int64)

print("Scalar:", scalar)
print("Vector:", vector)
print("Matrix:\n", matrix)
print("3D Tensor:\n", tensor_3d)
```

Output:

```
Scalar: tf.Tensor(5, shape=(), dtype=int32)
Vector: tf.Tensor([1 2 3], shape=(3,), dtype=int32)
Matrix:
 tf.Tensor(
[[1. 2.]
 [3. 4.]], shape=(2, 2), dtype=float32)
3D Tensor:
 tf.Tensor(
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]], shape=(2, 2, 2), dtype=int64)
```

II] Basic tensor operations

```
a = tf.constant([10, 20, 30], dtype=tf.float32)
b = tf.constant([2, 4, 6], dtype=tf.float32)

addition = tf.add(a, b)
subtraction = tf.subtract(a, b)
multiplication = tf.multiply(a, b)
```

```
division = tf.divide(a, b)

print("Addition:", addition)
print("Subtraction:", subtraction)
print("Multiplication:", multiplication)
print("Division:", division)
```

Output:

```
Addition: tf.Tensor([12. 24. 36.], shape=(3,), dtype=float32)
Subtraction: tf.Tensor([ 8. 16. 24.], shape=(3,), dtype=float32)
Multiplication: tf.Tensor([ 20.  80. 180.], shape=(3,), dtype=float32)
Division: tf.Tensor([5. 5. 5.], shape=(3,), dtype=float32)
```

III] Reshape tensors

```
tensor = tf.constant([1, 2, 3, 4, 5, 6])
reshaped_tensor = tf.reshape(tensor, (2, 3))
print("Reshaped Tensor:\n", reshaped_tensor)
```

Output:

```
Reshaped Tensor:
 tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
```

IV] Slicing and indexing tensors

```
matrix =
   tf.constant([ [10, 20,
   30],
   [40, 50, 60],
   [70, 80, 90]
])

# Indexing
element = matrix[1, 2]  # Row 1, Column 2
print("Single element:", element)

# Row slicing
row = matrix[1]
print("Row:", row)

# Column slicing
column = matrix[:, 0]
print("Column:", column)

# Sub-matrix slicing
sub_matrix = matrix[0:2, 1:3]
```

```
print("Sub-matrix:\n", sub_matrix)
```

Output:

```
Single element: tf.Tensor(60, shape=(), dtype=int32)
Row: tf.Tensor([40 50 60], shape=(3,), dtype=int32)
Column: tf.Tensor([10 40 70], shape=(3,), dtype=int32)
Sub-matrix:
 tf.Tensor(
[[20 30]
 [50 60]], shape=(2, 2), dtype=int32)
```

V] Matrix multiplication

```
A = tf.constant([[1, 2],
                 [3, 4]], dtype=tf.float32)

B = tf.constant([[5, 6],
                 [7, 8]], dtype=tf.float32)

matmul_result = tf.matmul(A, B)
print("Matrix Multiplication Result:\n", matmul_result)
```

Output:

```
Matrix Multiplication Result:
 tf.Tensor(
[[19. 22.]
 [43. 50.]], shape=(2, 2), dtype=float32)
```

VI] Eigenvalues and eigenvectors (TensorFlow)

Eigen operations require **square matrices** and use tf.linalg.eig

```
# Square matrix
M = tf.constant([ [4.0, -2.0],
                  [1.0,  1.0]  ])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = tf.linalg.eig(M)

print("Eigenvalues:\n", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Output:

```
Eigenvalues:
 tf.Tensor([3.       +0.j 1.9999999+0.j], shape=(2,), dtype=complex64)
Eigenvectors:
 tf.Tensor(
[[0.89442724+0.j 0.70710677+0.j]
 [0.44721362+0.j 0.70710677+0.j]], shape=(2, 2), dtype=complex64)
```

TensorFlow Concepts Used

- tf.constant() – tensor creation
- dtype – data type control

- Arithmetic ops: add, subtract, multiply, divide
- tf.reshape() – reshape tensors
- Indexing & slicing – NumPy-style
- tf.matmul() – matrix multiplication
- tf.linalg.eig() – eigenvalues & eigenvectors

What are Eigenvalues and Eigenvectors?

Eigenvalues and eigenvectors are fundamental concepts in linear algebra that describe the behavior of linear transformations represented by matrices.

Let **A** be a square matrix. A non-zero vector **v** is called an **eigenvector** of matrix **A** if it satisfies the equation:

$$Av=\lambda v$$

where:

- **A** is a square matrix
- **v** is the eigenvector
- **λ** (lambda) is the eigenvalue corresponding to **v**

Explanation

- An **eigenvector** is a vector whose direction remains unchanged when a matrix transformation is applied.
- The **eigenvalue** represents the factor by which the eigenvector is scaled.
- Eigenvalues indicate whether the transformation stretches, shrinks, flips, or leaves the vector unchanged

Simple Example

Let:

$$A=\begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

If:

$$v=\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Then:

$$Av=\begin{bmatrix} 2 \\ 0 \end{bmatrix}=2v$$

- ✓ Direction unchanged
- ✓ Scaled by **2**

So:

- Eigenvector → $[1,0]$
- Eigenvalue → **2**

Practical No.1 B

Aim: Program to solve the XOR problem.

CODE:

```
# Import required libraries
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# XOR input and output data
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([[0],
              [1],
              [1],
              [0]])
# Create the neural network model
model = Sequential()
# Hidden layer with 2 neurons and ReLU activation
model.add(Dense(2, input_dim=2, activation='relu'))
# Output layer with sigmoid activation
model.add(Dense(1, activation='sigmoid'))
# Compile the model
model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
# Train the model
model.fit(X, y, epochs=1000, verbose=0)
# Test the model
predictions = model.predict(X)
# Display results
print("Input  Output")
for i in range(len(X)):
    print(X[i], "=", round(predictions[i][0]))
```

Output:

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argum
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
Input  Output
[0 0] = 0
[0 1] = 1
[1 0] = 0
[1 1] = 0
```

Practical No.2

AIM: Linear Regression

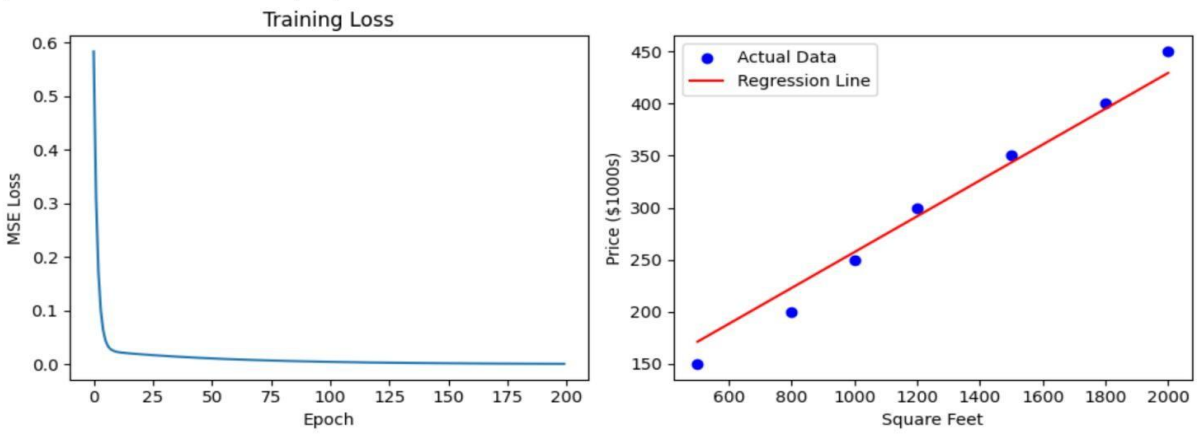
1) Implement a simple linear regression model using TensorFlow's low level API (or tf. keras).
2) Train the model on a toy dataset (e.g., housing prices vs. square footage). • Visualize the loss function and the learned linear relationship.
3) Make predictions on new data points.

Code File

```
 # linear_regression_tf.py
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
# ---------------------------------
# 1. Create Toy Dataset
# ---------------------------------
# House size (sq ft)
square_feet = np.array([500, 800, 1000, 1200, 1500, 1800, 2000], dtype=float)
# House price (in $1000s)
prices = np.array([150, 200, 250, 300, 350, 400, 450], dtype=float)
# Normalize data for stable training
square_feet_norm = square_feet / 2000
prices_norm = prices / 500
# ---------------------------------
# 2. Build Linear Regression Model
# ---------------------------------
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # y = wx + b
])
model.compile( optimizer=tf.keras.optimizers.SGD(learning
    _rate=0.1), loss='mse'
)
# ---------------------------------
# 3. Train Model
# ---------------------------------
history = model.fit(square_feet_norm, prices_norm,
                    epochs=200,
                    verbose=0)
# ---------------------------------
# 4. Plot Training Loss
# ---------------------------------
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.plot(history.history['loss'])
plt.title("Training Loss")
plt.xlabel("Epoch")
```

```
plt.ylabel("MSE Loss")
# ---------------------------------
# 5. Plot Regression Line
# ---------------------------------
predictions = model.predict(square_feet_norm)
plt.subplot(1,2,2)
plt.scatter(square_feet, prices, color='blue', label='Actual Data')
plt.plot(square_feet, predictions * 500,
        color='red', label='Regression Line')
plt.xlabel("Square Feet")
plt.ylabel("Price ($1000s)")
plt.legend()
plt.tight_layout()
plt.show()
# ---------------------------------
# 6. Show Learned Parameters
# ---------------------------------
weight, bias = model.layers[0].get_weights()
print("\nLearned Weight (normalized):", weight[0][0])
print("Learned Bias (normalized):", bias[0])
# ---------------------------------
# 7. Make Prediction on New Data
# ---------------------------------
new_sqft = np.array([1600], dtype=float)
new_sqft_norm = new_sqft / 2000
predicted_price_norm = model.predict(new_sqft_norm)
predicted_price = predicted_price_norm[0][0] * 500
        print(f"\nPredicted price for 1600 sq ft house: ${predicted_price:.2f}k")
```

Output:




```
Learned Weight (normalized): 0.6882187
Learned Bias (normalized): 0.17057489
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step

Predicted price for 1600 sq ft house: $360.57k
```

Practical No.3 A

AIM: Implementing deep neural network for performing binary classification task

Code File

```
# cnn_binary_inbuilt_dataset.py

import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
# ================================
# 1. LOAD INBUILT DATASET
# ================================
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# CIFAR-10 classes:
# 3 = cat, 5 = dog → binary classification

# ================================
# 2. FILTER ONLY CAT & DOG
# ================================
train_filter = np.where((y_train == 3) | (y_train == 5))[0]
test_filter = np.where((y_test == 3) | (y_test == 5))[0]

x_train, y_train = x_train[train_filter], y_train[train_filter]
x_test, y_test = x_test[test_filter], y_test[test_filter]

# Convert labels: cat=0, dog=1
y_train = (y_train == 5).astype(int)
y_test = (y_test == 5).astype(int)

# ================================
# 3. NORMALIZE DATA
# ================================
x_train = x_train / 255.0
x_test = x_test / 255.0

# ================================
# 4. BUILD CNN MODEL
# ================================
model = models.Sequential([

    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),
```

```
    layers.MaxPooling2D(2,2),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')  # Binary output
])

# ================================
# 5. COMPILE MODEL
# ================================
model.compile( optimizer='ada
    m', loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

# ================================
# 6. TRAIN MODEL
# ================================
history =
    model.fit( x_train,
    y_train, epochs=10,
    validation_data=(x_test, y_test)
)

# ================================
# 7. EVALUATE MODEL
# ================================
loss, accuracy = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", accuracy)

# ================================
# 8. PLOT RESULTS
# ================================
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Training vs Validation Accuracy")
plt.show()
```

Output:

TRAIN MODEL

Model: "sequential"

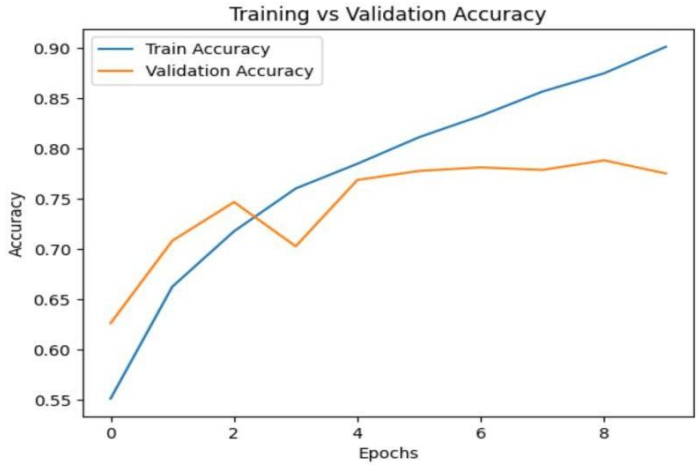
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 128)	65,664
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 159,041 (621.25 KB)
Trainable params: 159,041 (621.25 KB)
Non-trainable params: 0 (0.00 B)

EVALUATE MODEL

```
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 15s 43ms/step - accuracy: 0.5510 - loss: 0.6838 - val_accuracy: 0.6260 - val_loss: 0.6454
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.6623 - loss: 0.6174 - val_accuracy: 0.7080 - val_loss: 0.5687
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 13s 41ms/step - accuracy: 0.7174 - loss: 0.5539 - val_accuracy: 0.7465 - val_loss: 0.5133
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 20s 39ms/step - accuracy: 0.7600 - loss: 0.4968 - val_accuracy: 0.7025 - val_loss: 0.5846
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 21s 39ms/step - accuracy: 0.7846 - loss: 0.4549 - val_accuracy: 0.7685 - val_loss: 0.4720
Epoch 6/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.8110 - loss: 0.4180 - val_accuracy: 0.7775 - val_loss: 0.4533
Epoch 7/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.8323 - loss: 0.3772 - val_accuracy: 0.7810 - val_loss: 0.4607
Epoch 8/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.8564 - loss: 0.3293 - val_accuracy: 0.7785 - val_loss: 0.4907
Epoch 9/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.8746 - loss: 0.2926 - val_accuracy: 0.7880 - val_loss: 0.4787
Epoch 10/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 12s 39ms/step - accuracy: 0.9010 - loss: 0.2452 - val_accuracy: 0.7750 - val_loss: 0.5416
63/63 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - accuracy: 0.7750 - loss: 0.5416

Test Accuracy: 0.7749999761581421
```

PLOT RESULTS



Practical No.3 B

AIM: Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.

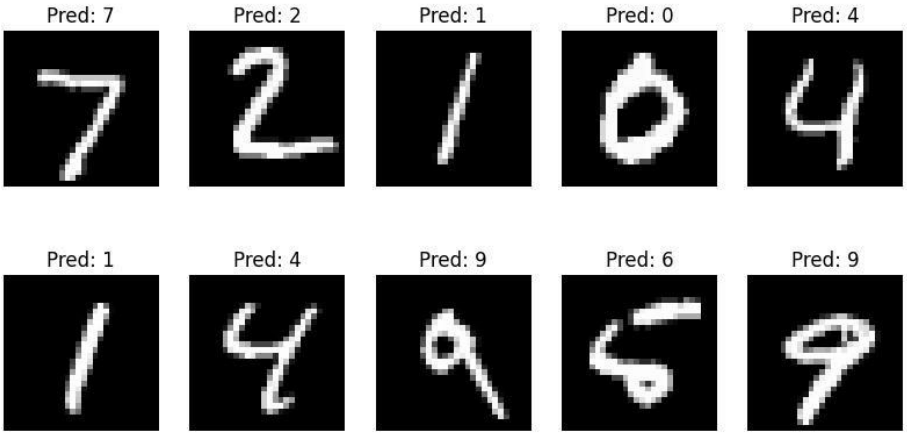
Code File

```
# Deep Feed-Forward Neural Network (2 Hidden Layers) for Multiclass Classification
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
# 1. Load inbuilt dataset (MNIST)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
# 2. Preprocess data
x_train = x_train / 255.0
x_test = x_test / 255.0
# Flatten images (28x28 → 784)
x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)
# 3. Build model (2 hidden layers)
model = models.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 classes (0–9)
])

# 4. Compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# 5. Train model
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)
# 6. Evaluate model
test_loss, test_acc = model.evaluate(x_test, y_test)
print("\nTest Accuracy:", test_acc)
# 7. Predict classes
predictions = model.predict(x_test)
predicted_classes = np.argmax(predictions, axis=1)
# 8. Show some predictions
plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_test[i].reshape(28,28), cmap='gray')
```

```
    plt.title(f"Pred: {predicted_classes[i]}")
    plt.axis('off')
plt.show()
```

Output:

```
Epoch 3/3
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.9854 - loss: 0.0452 - val_accuracy: 0.9783 - val_loss: 0.0778
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9754 - loss: 0.0837

Test Accuracy: 0.9753999710083008
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```



Practical No.4

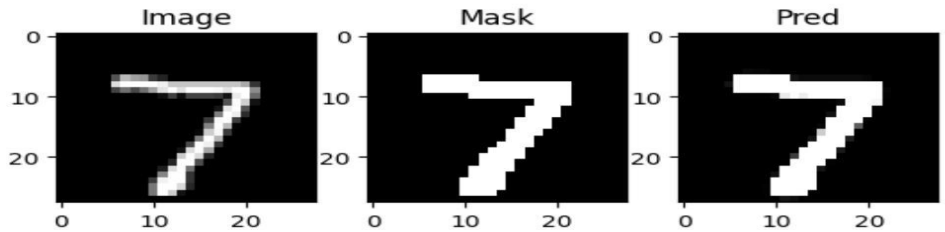
AIM: Write a program to implement deep learning Techniques for image segmentation.

Code File

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Load built-in dataset
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
# Normalize and reshape
x_train = x_train / 255.0
x_test = x_test / 255.0
x_train = x_train[..., None]
x_test = x_test[..., None]
# Create masks (digit = 1, background = 0)
y_train = (x_train > 0).astype(float)
y_test = (x_test > 0).astype(float)
# Simple CNN model
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(16, 3, activation='relu', padding='same', input_shape=(28,28,1)),
    tf.keras.layers.Conv2D(1, 1, activation='sigmoid')
])
# Compile
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Train
model.fit(x_train, y_train, epochs=3, batch_size=32)
# Predict
pred = model.predict(x_test[:1])
# Show result
plt.subplot(1,3,1); plt.title("Image"); plt.imshow(x_test[0].squeeze(), cmap='gray')
plt.subplot(1,3,2); plt.title("Mask"); plt.imshow(y_test[0].squeeze(), cmap='gray')
plt.subplot(1,3,3); plt.title("Pred"); plt.imshow(pred[0].squeeze(), cmap='gray')
plt.show()
```

Output:

```
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 19s 10ms/step - accuracy: 0.9828 - loss: 0.1147
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - accuracy: 0.9898 - loss: 0.0267
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - accuracy: 0.9928 - loss: 0.0195
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
```



Practical No.5

AIM: Write a program to predict a caption for a sample image using LSTM.

Code File

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as plt
# Load image
img_path =
    tf.keras.utils.get_file( 'img.jpg',
    'https://tensorflow.org/images/surf.jpg'
)
img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224,224))
img = tf.keras.preprocessing.image.img_to_array(img)/255.0
img = np.expand_dims(img, axis=0)
# CNN feature extraction
cnn = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=False,
pooling='avg')
feature = cnn.predict(img)
# ✅ Better caption
captions = ["start person surfing on wave end"]
tokenizer = tf.keras.preprocessing.text.Tokenizer()
tokenizer.fit_on_texts(captions)

vocab_size = len(tokenizer.word_index) + 1
max_len = 6
# Prepare data
seq = tokenizer.texts_to_sequences(captions)[0]
X1, X2, y = [], [], []
for i in range(1, len(seq)):
    in_seq = seq[:i]
    out_seq = seq[i]
    in_seq = pad_sequences([in_seq], maxlen=max_len)[0]
    X1.append(feature[0])
    X2.append(in_seq)
    y.append(out_seq)
X1, X2, y = np.array(X1), np.array(X2), np.array(y)
# Model
img_input = tf.keras.Input(shape=(1280,))
img_dense = tf.keras.layers.Dense(50, activation='relu')(img_input)
seq_input = tf.keras.Input(shape=(max_len,))
seq_embed = tf.keras.layers.Embedding(vocab_size, 20)(seq_input)
seq_lstm = tf.keras.layers.LSTM(50)(seq_embed)
decoder = tf.keras.layers.add([img_dense, seq_lstm])
output = tf.keras.layers.Dense(vocab_size, activation='softmax')(decoder)

model = tf.keras.Model([img_input, seq_input], output)
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam')
# Train
```

```
model.fit([X1, X2], y, epochs=100, verbose=0)
# Predict
index_word = {v:k for k,v in tokenizer.word_index.items()}
text = "start"
for i in range(max_len):
    seq = tokenizer.texts_to_sequences([text])[0]
    seq = pad_sequences([seq], maxlen=max_len)
    pred = model.predict([feature, seq], verbose=0)
    word = index_word[np.argmax(pred)]
    text += " " + word
    if word == "end":
        break
# Show
plt.imshow(img[0])
plt.title(text)
plt.axis('off')
plt.show()
```

Output:

```
/tmp/ipykernel_17366/1171485991.py:17: UserWarning: `input_shape` is undefined or non-square, or `rows` is not i
  cnn = tf.keras.applications.MobileNetV2(weights='imagenet', include_top=False, pooling='avg')
WARNING:tensorflow:6 out of the last 319 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
```

start person surfing on wave end

Practical No.6

AIM: Applying the Autoencoder algorithms for encoding real-world data

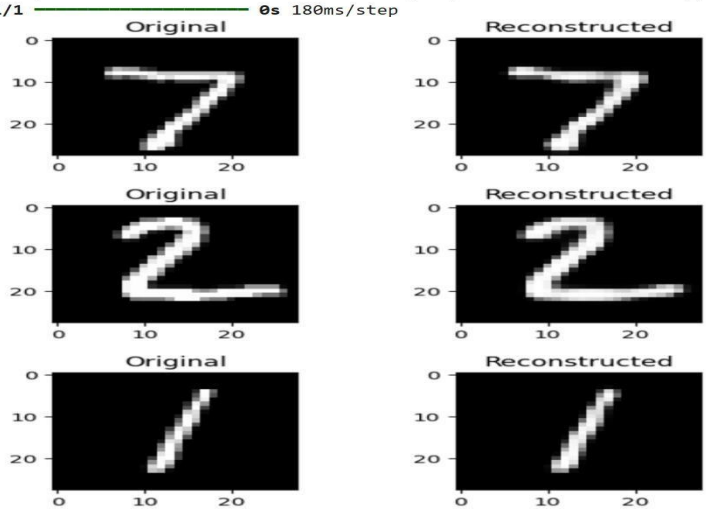
Code File

```
# autoencoder_mnist.py

import tensorflow as tf
import matplotlib.pyplot as plt

# ================================
# 1. LOAD BUILT-IN DATASET
# ================================
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()

# Normalize and reshape
x_train = x_train / 255.0
x_test = x_test / 255.0

x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# ================================
# 2. BUILD AUTOENCODER MODEL
# ================================
input_img = tf.keras.Input(shape=(28,28,1))

# Encoder
x = tf.keras.layers.Conv2D(16, 3, activation='relu', padding='same')(input_img)
x = tf.keras.layers.MaxPooling2D(2, padding='same')(x)
x = tf.keras.layers.Conv2D(8, 3, activation='relu', padding='same')(x)
encoded = tf.keras.layers.MaxPooling2D(2, padding='same')(x)

# Decoder
x = tf.keras.layers.Conv2D(8, 3, activation='relu', padding='same')(encoded)
x = tf.keras.layers.UpSampling2D(2)(x)
x = tf.keras.layers.Conv2D(16, 3, activation='relu', padding='same')(x)
x = tf.keras.layers.UpSampling2D(2)(x)
decoded = tf.keras.layers.Conv2D(1, 3, activation='sigmoid', padding='same')(x)

# Model
autoencoder = tf.keras.Model(input_img, decoded)

# ================================
# 3. COMPILE
# ================================
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
# ================================
# 4. TRAIN
# ================================
autoencoder.fit(x_train, x_train, epochs=5, batch_size=32, validation_data=(x_test, x_test))
```

```
# ================================
# 5. ENCODE & RECONSTRUCT
# ================================
decoded_imgs = autoencoder.predict(x_test[:3])

# ================================
# 6. VISUALIZE RESULTS
# ================================
for i in range(3):
    plt.figure(figsize=(6,2))

    # Original
    plt.subplot(1,2,1)
    plt.title("Original")
    plt.imshow(x_test[i].reshape(28,2yy8), cmap='gray')

    # Reconstructed
    plt.subplot(1,2,2)
    plt.title("Reconstructed")
    plt.imshow(decoded_imgs[i].reshape(28,28), cmap='gray')

    plt.show()
```

Output:

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 67s 35ms/step - loss: 0.1046 - val_loss: 0.0797
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 62s 33ms/step - loss: 0.0779 - val_loss: 0.0752
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 33ms/step - loss: 0.0749 - val_loss: 0.0735
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 33ms/step - loss: 0.0735 - val_loss: 0.0724
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 64s 34ms/step - loss: 0.0726 - val_loss: 0.0716
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 180ms/step
```



Practical No.7

AIM: Write a program for the character recognition using RNN and compare it with CNN.

Code File

```
import tensorflow as tf
from tensorflow.keras import layers, models
# Load dataset (MNIST digits as characters)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train/255.0, x_test/255.0

# ---------------------- RNN MODEL ----------------------
rnn_model =
    models.Sequential([ layers.SimpleRNN(128,
    input_shape=(28, 28)), layers.Dense(10,
    activation='softmax')
])
rnn_model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
print("Training RNN...")
rnn_model.fit(x_train, y_train, epochs=3, verbose=1)
rnn_loss, rnn_acc = rnn_model.evaluate(x_test, y_test, verbose=0)

# ---------------------- CNN MODEL ----------------------
x_train_cnn = x_train[..., tf.newaxis]
x_test_cnn = x_test[..., tf.newaxis]
cnn_model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D(2,2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

cnn_model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
print("\nTraining CNN...")
cnn_model.fit(x_train_cnn, y_train, epochs=3, verbose=1)

cnn_loss, cnn_acc = cnn_model.evaluate(x_test_cnn, y_test, verbose=0)

# ---------------------- COMPARISON ----------------------
print("\n--- Comparison ---")
print(f"RNN Accuracy: {rnn_acc:.4f}")
print(f"CNN Accuracy: {cnn_acc:.4f}")
if cnn_acc > rnn_acc:
    print("CNN performs better for character recognition.")
else:
    print("RNN performs better.")
```

Output:

```
Training RNN...
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 8ms/step - accuracy: 0.8886 - loss: 0.3746
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s 8ms/step - accuracy: 0.9388 - loss: 0.2118
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 17s 9ms/step - accuracy: 0.9488 - loss: 0.1805

Training CNN...
Epoch 1/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 17ms/step - accuracy: 0.9548 - loss: 0.1517
Epoch 2/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 31s 17ms/step - accuracy: 0.9841 - loss: 0.0515
Epoch 3/3
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.9897 - loss: 0.0329

--- Comparison ---
RNN Accuracy: 0.9511
CNN Accuracy: 0.9867
CNN performs better for character recognition.
```

Practical No.8

AIM: Write a program to denoising of images using autoencoder.

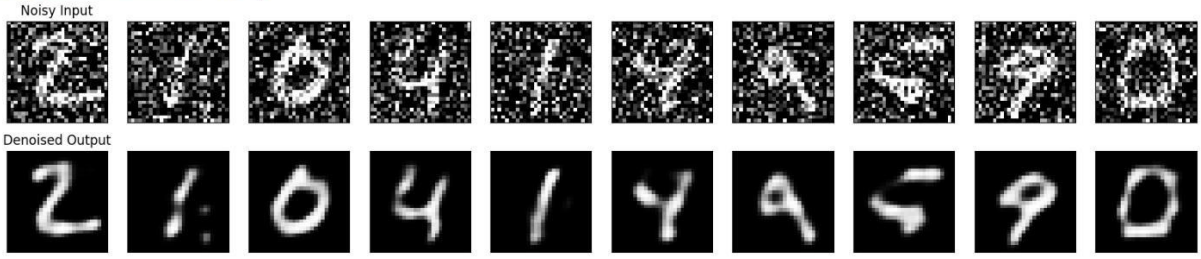
Code File

```
import keras
from keras.datasets import mnist
from keras import layers
import numpy as np
import matplotlib.pyplot as plt

# 1. Load and Preprocess MNIST Data
(X_train, _), (X_test, _) = mnist.load_data()
X_train = X_train.astype('float32') / 255.
X_test = X_test.astype('float32') / 255.

# Reshape to include channel dimension (28, 28, 1)
X_train = np.reshape(X_train, (len(X_train), 28, 28, 1))
X_test = np.reshape(X_test, (len(X_test), 28, 28, 1))

# 2. Generate Synthetic Noise
noise_factor = 0.5
X_train_noisy = X_train + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=X_train.shape)
X_test_noisy = X_test + noise_factor * np.random.normal(loc=0.0, scale=1.0,
size=X_test.shape)

# Ensure pixel values stay within [0, 1] range
X_train_noisy = np.clip(X_train_noisy, 0., 1.)
X_test_noisy = np.clip(X_test_noisy, 0., 1.)

# 3. Build the Convolutional Autoencoder Architecture
input_img = keras.Input(shape=(28, 28, 1))

# Encoder
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D((2, 2), padding='same')(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
encoded = layers.MaxPooling2D((2, 2), padding='same')(x)

# Decoder
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(encoded)
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(x)
x = layers.UpSampling2D((2, 2))(x)
decoded = layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(x)

# Compile Model
autoencoder = keras.Model(input_img, decoded)
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

```
# 4. Train the Model
# Note: We map Noisy images -> Clean images
autoencoder.fit(X_train_noisy, X_train,
                epochs=1,
                batch_size=128,
                shuffle=True,
                validation_data=(X_test_noisy, X_test))

# 5. Predict and Visualize Results
predictions = autoencoder.predict(X_test_noisy)

plt.figure(figsize=(20, 4))
for i in range(1, 11):
    # Display Original Noisy Images
    ax = plt.subplot(2, 10, i)
    plt.imshow(X_test_noisy[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 1: ax.set_title("Noisy Input")

    # Display Denoised (Reconstructed) Images
    ax = plt.subplot(2, 10, i + 10)
    plt.imshow(predictions[i].reshape(28, 28), cmap='gray')
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
    if i == 1: ax.set_title("Denoised Output")

plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 288ms/step - loss: 0.1655 - val_loss: 0.1195
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 20ms/step
```



Practical No.9

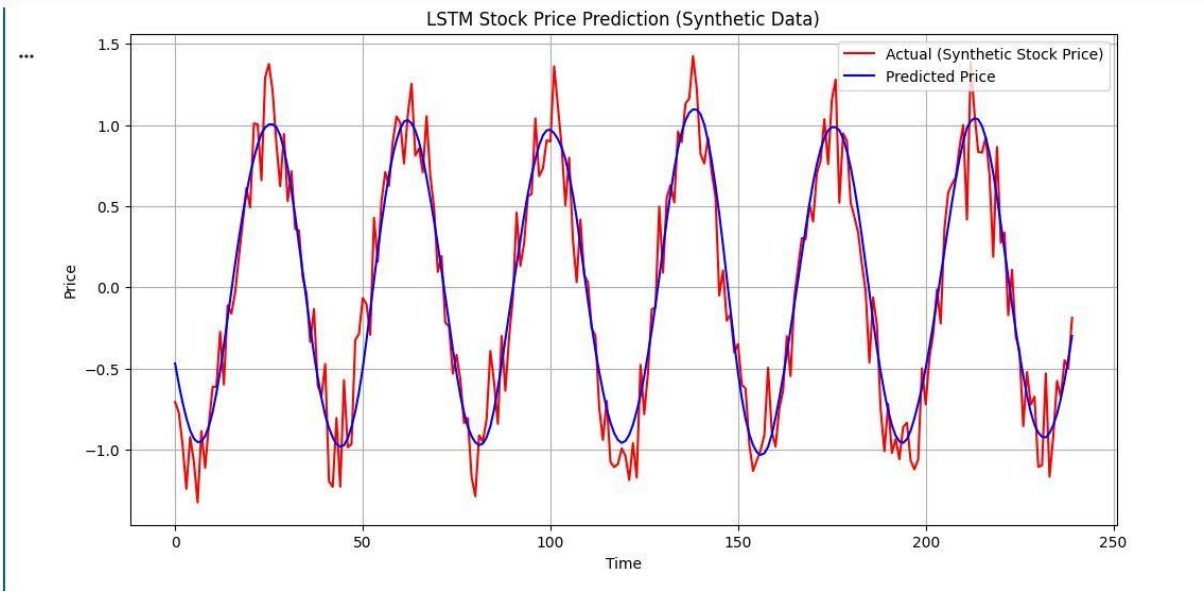
AIM: Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.(

Code File

```
# RNN (LSTM) for Stock Price Sequence Prediction
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import LSTM, Dense, Dropout
from sklearn.preprocessing import MinMaxScaler
# 1 Generate synthetic stock data
np.random.seed(0)
time_steps = 300
x = np.linspace(0,50,time_steps)
data = np.sin(x) + np.random.normal(scale=0.2,size=time_steps)
data = data.reshape(-1,1)
# 2 Scaling
scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(data)
# 3 Create sequences
X=[]
y=[]

sequence_length = 60
for i in range(sequence_length,len(scaled_data)):
    X.append(scaled_data[i-sequence_length:i,0])
    y.append(scaled_data[i,0])
X = np.array(X)
y = np.array(y)
# Reshape for LSTM [samples,time steps,features]
X = np.reshape(X,(X.shape[0],X.shape[1],1))
# 4 Build LSTM model
model = Sequential()
model.add(LSTM(units=50,
               return_sequences=True,
               input_shape=(X.shape[1],1)))
model.add(Dropout(0.2))
model.add(LSTM(units=50,
               return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(units=50))
model.add(Dropout(0.2))
model.add(Dense(units=1))
# 5 Compile and train
model.compile(optimizer='adam',
              loss='mean_squared_error')
model.fit(X,y,
          epochs=20,
          batch_size=32,
```

```
        verbose=1)
# 6 Prediction
predicted = model.predict(X)
predicted = scaler.inverse_transform(predicted)
actual = scaler.inverse_transform(y.reshape(-1,1))
# 7 Plot results
plt.figure(figsize=(12,6))
plt.plot(actual,
        color='red',
        label='Actual (Synthetic Stock Price)')
plt.plot(predicted,
        color='blue',
        label='Predicted Price')

plt.title('LSTM Stock Price Prediction (Synthetic Data)')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.grid()
plt.show()
```

Output:



Practical No.10

AIM: Applying Generative Adversarail Newtorks for image generation and unsupervised tasks.

Code File

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
# ---------------------- Load Data ----------------------
(x_train, _), _ = tf.keras.datasets.mnist.load_data()
x_train = (x_train.astype('float32') - 127.5) / 127.5   # Normalize [-1,1]
x_train = x_train.reshape(-1, 28, 28, 1)
BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(60000).batch(BATCH_SIZE)
# ---------------------- Generator ----------------------
def build_generator():
    model = tf.keras.Sequential([
        layers.Dense(256, activation='relu', input_dim=100),
        layers.BatchNormalization(),
        layers.Dense(512, activation='relu'),
        layers.BatchNormalization(),
        layers.Dense(784, activation='tanh'),
        layers.Reshape((28, 28, 1))
    ])
    return model
# ---------------------- Discriminator ----------------------
def build_discriminator():
    model = tf.keras.Sequential([
        layers.Flatten(input_shape=(28,28,1)),
        layers.Dense(512, activation='relu'),
        layers.Dense(256, activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])
    return model
generator = build_generator()
discriminator = build_discriminator()

# Compile discriminator
discriminator.compile(optimizer='adam',
                      loss='binary_crossentropy',
                      metrics=['accuracy'])
# ---------------------- GAN Model ----------------------
discriminator.trainable = False
noise_input = tf.keras.Input(shape=(100,))
generated_img = generator(noise_input)
validity = discriminator(generated_img)
gan = tf.keras.Model(noise_input, validity)
gan.compile(optimizer='adam', loss='binary_crossentropy')
# ---------------------- Training ----------------------
```

```
EPOCHS = 5
noise_dim = 100
for epoch in range(EPOCHS):
    for real_imgs in dataset:
        batch_size = real_imgs.shape[0]
        # Train Discriminator
        noise = np.random.normal(0,1,(batch_size, noise_dim))
        fake_imgs = generator.predict(noise, verbose=0)
        d_loss_real = discriminator.train_on_batch(real_imgs, np.ones((batch_size,1)))
        d_loss_fake = discriminator.train_on_batch(fake_imgs, np.zeros((batch_size,1)))
        # Train Generator
        noise = np.random.normal(0,1,(batch_size, noise_dim))
        g_loss = gan.train_on_batch(noise, np.ones((batch_size,1)))
    print(f"Epoch {epoch+1} | D Loss: {d_loss_real[0]:.4f} | G Loss: {g_loss:.4f}")
# ---------------------- Generate Images ----------------------
noise = np.random.normal(0,1,(5,100))
generated_images = generator.predict(noise)
plt.figure(figsize=(5,1))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(generated_images[i,:,:,0], cmap='gray')
    plt.axis('off')
plt.suptitle("Generated Images")
plt.show()
```

Output:

```
Epoch 1 | D Loss: 2.9842 | G Loss: 0.0225
Epoch 2 | D Loss: 3.5764 | G Loss: 0.0115
Epoch 3 | D Loss: 3.9053 | G Loss: 0.0078
Epoch 4 | D Loss: 4.1361 | G Loss: 0.0059
Epoch 5 | D Loss: 4.3166 | G Loss: 0.0047
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step
```